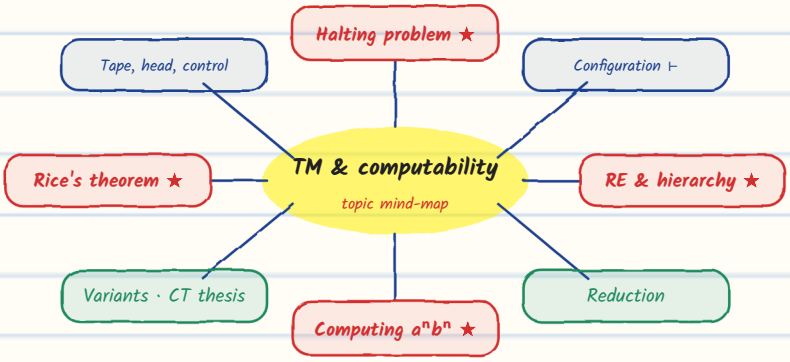
Date: 14.10.2026
topper's notes

Turing Machines & Computability Theory

7-tuple · configuration ⊢ · computing $a^n b^n$ · halting problem · reduction · Rice · RE & hierarchy: in 11 sheets



↑ the whole topic on one page: red = high-yield, green = method / applied

➜ Syllabus coverage

"... Turing Machines: basic machines, configuration, computing with Turing machines." That whole line is this notebook, plus the computability results the exam keeps asking about.

High-yield list

① TM = 7-tuple (Q, Σ, Γ, δ, q0, B, F)
② config = left, state, right; ⊢ = one move
③ halting problem undecidable (1936), yet RE
④ multi-tape = speed, never power
⑤ Rice: non-trivial property of L(M)
⑥ Regular ⊂ CF ⊂ CS ⊂ Recursive ⊂ RE

Legend yellow = key term · red circle = must-memorise · **purple** = trick · **red** = trap · **green** = example

➜ Contents

✓ What a Turing machine is ..... p. 2
✓ Configuration & the move ⊢ ..... p. 3
✓ Computing with TMs: the $a^n b^n$ machine ..... p. 4
✓ Variants, Church-Turing, LBAs ..... p. 5
✓ Decidable · semi · undecidable ..... p. 6
✓ The halting problem ..... p. 7
✓ Reduction and Rice's theorem ..... p. 8
✓ RE, Co-RE & the hierarchy ..... p. 9
✓ Night-before sheet ..... p. 10
✓ Practice MCQs + written ..... p. 11

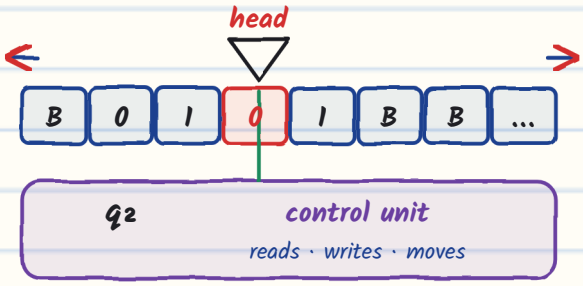
1 What a Turing Machine Is

high-yield

Definition A **Turing machine (TM)** is the most powerful computational model: a finite control unit driving a read/write head over an unbounded tape divided into cells. The tape has no end, so a TM can simulate **any** algorithm — and it recognises exactly the recursively enumerable languages.

→ The 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$

Part	Is	In this machine
Q	finite set of **states**	$Q = \{q_0, q_1, q_2\}$
Σ	**input** alphabet; $B \notin \Sigma$	$\Sigma = \{0, 1\}$
Γ	**tape** alphabet; $\Sigma \subseteq \Gamma$, $B \in \Gamma$	$\Gamma = \{0, 1, B\}$
δ	$Q \times \Gamma \to Q \times \Gamma \times \{L, R, S\}$	$\delta(q_0, 0) = (q_1, 1, R)$
q_0	the **start state**, $q_0 \in Q$	q_0
B	the **blank symbol**, $B \in \Gamma \setminus \Sigma$	B
F	set of **accepting** states, $F \subseteq Q$	$F = \{q_2\}$



tape is infinite in both directions

the signature picture: tape, head, finite control

Transition format δ(state, symbol read) = (new state, symbol written, direction). So $\delta(q_0, 0) = (q_1, 1, R)$ reads: *while in q_0, reading 0, write 1, move Right, enter q_1.* One transition = one step — the whole machine is this table plus the 7-tuple.

Exam trap The **general** form lets δ move the head **three** ways: $\{L, R, S\}$ where S = stay and write without moving. Some books print only $\{L, R\}$; that two-way machine is still universal, but calling $\{L, R\}$ the general definition loses a mark.

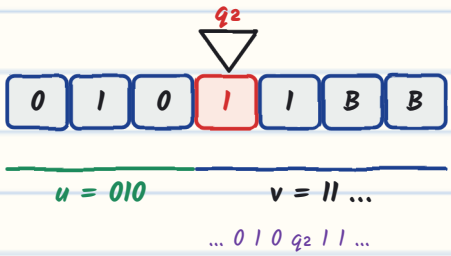
Memory trick "Q-states, Σ-input, Γ-tape, δ-steps, q_0-start, B-blank, F-final" — only the tape is unbounded, so memory comes from position.

2 Configuration: the Instantaneous Description

high-yield

Definition An **instantaneous description** (ID) records the machine's **complete** state at one instant, written **(u, q, v)**: **u** = tape contents **left** of the head, **q** = the current state, **v** = contents **from the head rightwards**, with the head scanning the **first symbol of v**.

Notation	Reads as
(u, q, v)	tape … u **q** v …, head on the first symbol of v
$(u, q, v) \vdash (u', q', v')$	the machine makes **one** move
$(u, q, v) \vdash^* (u', q', v')$	**zero or more** moves — reachability
$q_0 w$	the **start configuration** of the run on input w



the ID $(010, q_2, 11)$: head on the first symbol of v

Worked move ✍️

$(010, q_2, 11) \vdash (0101, q_3, 1)$ if $\delta(q_2, 1) = (q_3, 1, R)$

tape … 0 1 0 [1] 1 …
u = 010 ^q_2 v = 11

The head was on the first **1** of the right block; δ says write **1**, move **R**, go to q_3 — so the written **1** slides into **u**, the state changes, and the next symbol becomes the head of **v**.

Exam trap The symbol under the head belongs to **v**, never to **u**. Splitting as (01, q_2, 011) is **wrong** even though the same tape is described. And ⊢ is one step while $\vdash^*$ is many: swapping them changes the meaning of a trace completely.

Memory trick **left · state · right** — and the head "belongs" to the right-hand part. Say it aloud while writing (u, q, v) and the placement never slips.

Why the ID matters A TM has no separate input tape and no separate memory, so the tape **is** the whole configuration — two runs sharing an ID behave identically forever. That is exactly how you **prove a machine loops**: reach a repeated ID and the rest is periodic.

↙ $\vdash^*$ means reachability — "can get there", never "gets there in one move"

3 Computing with Turing Machines: the $a^n b^n$ Machine

written favourite

Worked machine ✍ $L = \{a^n b^n \mid n \geq 0\}$

$Q = \{q0, q1, q2, q3, q4\}$ Sigma $= \{a, b\}$ Gamma $= \{a, b, X, Y, B\}$

start $q0$ (marks an a, hunts right) accepting $F = \{q4\}$

δ(state, symbol)	What that move does
$\delta(q_0, a) = (q_1, X, R)$	mark the **first** a as **X**, move right
$\delta(q_1, a) = (q_1, a, R)$	skip the remaining a's
$\delta(q_1, Y) = (q_1, Y, R)$	skip the marked b's
$\delta(q_1, b) = (q_2, Y, L)$	mark that b as **Y** (one match), move left
$\delta(q_2, Y) = (q_2, Y, L)$	skip Y's on the way back
$\delta(q_2, a) = (q_2, a, L)$	skip a's on the way back
$\delta(q_2, X) = (q_0, X, R)$	first X reached ⇒ pair complete, restart
$\delta(q_0, Y) = (q_3, Y, R)$	no a left ⇒ all matched, check the b's
$\delta(q_3, Y) = (q_3, Y, R)$	skip the Y's
$\delta(q_3, B) = (q_4, B, R)$	blank reached, no loose b ⇒ **accept**

➜ Execution on "aabb"

start B a a b b B

step 1 B X a b b B mark the first a

step 2 B X a Y b B match it with a b

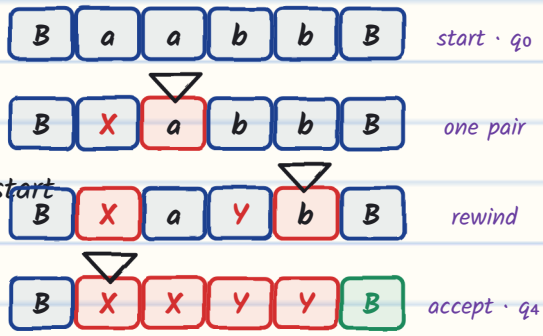
step 3 B X a Y b B head reached X, restart

step 4 B X X Y b B mark the second a

step 5 B X X Y Y B match it with a b

step 6 B X X Y Y B all a's matched

accept B X X Y Y B halt in q4



"aabb" marked up pair by pair, then accepted

It crosses out **one a with one b** per round trip, then rewinds. Run it on **ab** or **ba** and it never accepts.

Exam trap This table has **no rejecting transition** — there is no $\delta(q_0, B)$ and no $\delta(q_3, a)$. Feed it a non-member such as **abab** and it falls into an **undefined** transition instead of rejecting. A complete machine adds a rejecting state **q_rej** with $\delta(q_3, a) \rightarrow q_rej$ and $\delta(q_1, B) \rightarrow q_rej$.

Memory trick **"mark, hunt, match, rewind"** — q_0 marks, q_1 hunts the b, q_2 rewinds, q_3 verifies, q_4 accepts. X marks a's, Y marks b's.

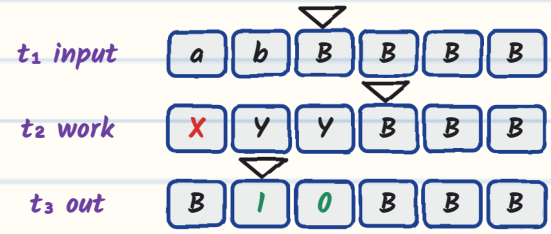
4 Variants of the TM and the Church-Turing Thesis

high-yield

Equivalence (high-yield)

Every variant below recognises exactly the same class of languages as a single-tape deterministic TM, with at worst a **polynomial** slowdown — a multi-tape machine costs at most a quadratic factor. **Extra tapes buy speed and convenience, never power.**

Variant	What it adds
Multi-tape TM	several tapes — input, work, output
Non-deterministic TM	several moves per state; accepts if **some** branch does
Multi-head TM	several heads on **one** tape
Offline TM	read-only input tape + work tapes + output
LBA	nondeterministic TM bounded by end markers



3 tapes ≡ 1 tape · same languages

a multi-tape machine: three heads, three tapes, one language class

Linear bounded automaton (LBA) Take a nondeterministic TM and cut its tape off with end markers ⊢ and ⊣, so the tape holds only the input plus the markers (**|w| + 2** cells). The head moves both ways inside that span but **cannot leave**. It accepts exactly the context-sensitive (Type-1) languages — this is where the CS layer of the hierarchy comes from.

Church-Turing thesis Turing-computable ⇔ algorithmically computable: every function computable by any algorithm can be computed by a TM, and vice versa. It is the licence to reason about "computable" using TMs.

Exam trap "Multi-tape TM is more powerful" is the classic wrong option — extra tapes are a **convenience**, at most **quadratic** slowdown on one tape. And a nondeterministic TM accepts iff **some** branch accepts, not all.

Memory trick The variants are the four ways to be generous with a machine — more tapes, more choices, more heads, separated I/O. Generosity changes **how fast**, never **what is recognisable**.

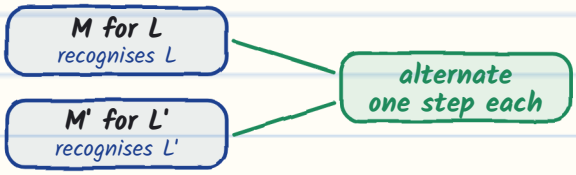
5 Decidable, Semi-Decidable, Undecidable

high-yield

The setting A **problem** asks a yes/no question about **every** input — infinitely many instances. Saying "yes to these three examples" settles nothing, so a problem is classified by **what its answering machine is allowed to do**: must it halt on everything, or may it run forever on the no-instances?

Class	What the machine does	Example
Decidable (recursive)	halts on **every** input, says yes or no	sorting, searching, $\{a^n b^n\}$ by a PDA
Semi-decidable (r.e. / RE)	halts and says **yes**; may **loop** on a no-instance	the halting problem, on its yes-instances
Undecidable	**no** algorithm answers it at all	halting; blank-tape halting; TM equivalence

The standard construction To **accept** an RE language, flip nothing: simulate M on w and accept when M accepts. To **decide** one, add a rejecting state and route **every** undefined transition there — then the machine halts on everything. Decidable is a strict subfamily of RE.



L and L' both RE ⇒ L recursive

exactly one of them must accept

L and its complement both RE ⇒ a decider exists

Memory trick A **decider** is a two-way machine: it can say no. A **recogniser** is one-way: it only confirms. Adding a reject state is the whole difference.

Why that last diagram is true If **L** and **L' = Σ* \ L** are both RE, run their two recognisers in parallel, alternating one step of each. For every input **exactly one** of M and M' must eventually accept — the complement relation guarantees no escape. Whichever accepts first settles the answer, and the other must accept eventually. That machine halts on everything, so **L is recursive**.

Exam trap **Undecidable does not mean "hard" or "slow"** — it means **no algorithm exists at all**. "Semi-decidable" is not a milder performance claim: the halting problem has a perfect yes-side procedure and is still undecidable.

Common misconception "Every computable function has a TM" is only half the statement. The other half is that every **non-computable** one provably has none — a TM cannot decide halting, count every computable real, or tell whether two programs behave alike.

6 The Halting Problem

the classic

Problem Given a Turing machine **M** and an input string **w**, decide whether **M** halts on **w** — written H(M, w) = yes or no. For programs the same question is: given **P** and **I**, does **P** stop on **I**?

Theorem (Turing, 1936) The halting problem is **undecidable** — no algorithm answers it for all M and w. It is nonetheless **recursively enumerable**: simulate M on w, and if M ever halts, say yes. A no-instance never produces an answer, which is exactly what "semi-decidable" means.

→ Proof by contradiction

1. **Assume** a decider **H(M, w)** exists that always halts and always answers correctly.
2. **Build D(M)**: a machine that runs M on input M and asks **H(M, M)**.
3. If **H says "halts"**, D **loops forever**.
4. If **H says "loops"**, D **halts immediately**.
5. **Run D on its own description**: D(D). If H said D halts, D loops; if H said D loops, D halts. **Either answer from H is wrong.**

The machine makes H answer its own question about itself, so H is refuted either way — and H was assumed correct on **every** pair. The trick that does it is the **diagonal**: run the supposed solver on the solver.

D is run on its own description

assumed decider H → D asks H(D, D)

H says halt ⇒ D loops
H says loop ⇒ D halts

whichever H says, it is wrong

the diagonal: D(D) refutes H either way

→ What it costs you

- ✓ **No general program analyser** — no tool can always tell a stuck loop from a slow computation.
- ✓ **No perfect debugger** — a linter cannot prove a function terminates for every input.
- ✓ **No all-purpose collector** — "is this value still reachable?" is undecidable in general.
- ✓ Anything that can **express** "does this stop?" inherits the impossibility.

Exam trap The halting problem is not "undecidable because it is infinite" — many infinite problems (sorting, $\{a^n b^n\}$ membership) are perfectly decidable. It is undecidable because of the **diagonal**: a logical obstruction, not a size one.

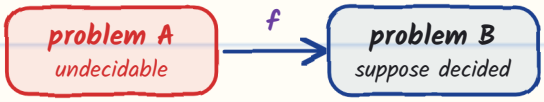
Memory trick Flip it. D does the **opposite** of what H predicts, so H is captured by its own answer. "Predict, then do the opposite" is the whole argument.

7 Reduction and Rice's Theorem

high-yield

Definition Problem **A many-one reduces** to problem **B**, written **A ≤ B**, if there is a computable function f such that for every input **w**: **w ∈ A ⇔ f(w) ∈ B**. In words — if you had a solver for B you could solve A by feeding it f(w). This is the workhorse of undecidability proofs.

If	Then
A is **undecidable** and A ≤ B	**B is undecidable**
B is **decidable** and A ≤ B	**A is decidable**



A ≤ B, A undecidable
⇒ B undecidable too

reduction: solve B, get A for free

➜ Reductions you will actually be asked for

Problem A	Reduces to
Halting M on w	halting on the **blank tape** — write w first
Halting M on w	does M ever **reach q_5** — make q_5 the halt state
Halting M on w	are **two TMs equivalent** — simulate M then N
CFG **ambiguity**	blank-tape halting, hence undecidable

Rice's theorem Let **P** be any **non-trivial property of the languages** recognised by Turing machines. Then **P is undecidable**. Precisely: **P** is a property of **L(M)** — the set of strings M accepts — not of M's description. **Non-trivial** means **some** machine's language has it and **some** does not.

➜ Undecidable — properties of L(M)

- ✓ Is **L(M) empty**? · **finite**? · **regular**?
- ✓ Is **L(M) = Σ***?
- ✓ Do **M and N** recognise the same language?

Exam trap Rice is about a property of the **language**. "Does M have 7 states?" is decidable by counting. "Is **L(M) infinite**?" is not — so to reduce, **encode the input into the machine**.

➜ Decidable — properties of the machine

- ✓ Does M ever **reach state q_5**?
- ✓ Does M **halt within 100 steps**?
- ✓ How many **states** does M have?

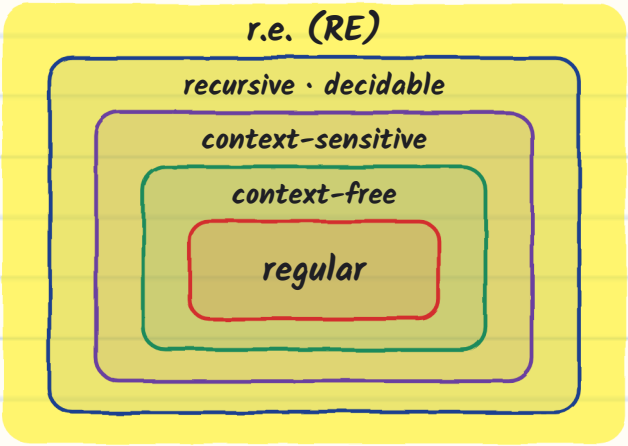
Memory trick **Rice asks about the set of strings, not the machine.** Write the question with L(M) in it, then ask whether it is non-trivial.

8 RE Languages, Co-RE and the Hierarchy high-yield

RE language A language some Turing machine **accepts** — equivalently, one for which a semi-decision procedure eventually confirms every member. A non-member need never be reached: the machine may loop on it forever. Every decidable language is RE; some RE languages are not decidable.

→ Closure properties of RE

Operation	RE closed?
union $L_1 \cup L_2$	yes — new start state, guess a branch
concatenation L_1L_2	yes — run M_1, then M_2
Kleene star L^*	yes — guess the split points
intersection $L_1 \cap L_2$	yes — alternate the two machines
complement L'	**no** — the recogniser may loop



Regular ⊂ CF ⊂ CS ⊂ Recursive ⊂ RE

the class hierarchy — each box nests inside the one above

→ One example per class

Class	Example
Regular	$\{w \mid w$ ends with $00\}$
Context-free	$\{a^n b^n \mid n \geq 0\}$ — one stack
Context-sensitive	$\{a^n b^n c^n \mid n \geq 1\}$ — bounded tape
Recursive	$\{w\#w \mid w \in \{0,1\}^*\}$ — copy and compare
r.e.	the halting problem (yes-instances)

RE vs Co-RE **Co-RE** = the **complements** of RE languages — problems a machine settles only on their **no**-instances. The halting problem is RE but **not** Co-RE; were it both, the parallel argument on sheet 6 would make it decidable, and it is not.

Exam trap "RE is closed under complement" is **false**, and that one fact explains three others: not every RE language is decidable, not every RE language is Co-RE, and some are **neither**. If **L and L' are both RE**, L is recursive

Memory trick Each layer buys exactly **one more resource**: automaton → one counter, stack → one nest, LBA → a bounded tape, TM → an **unbounded** tape.

Night-Before-the-Exam Sheet

Fact sheet

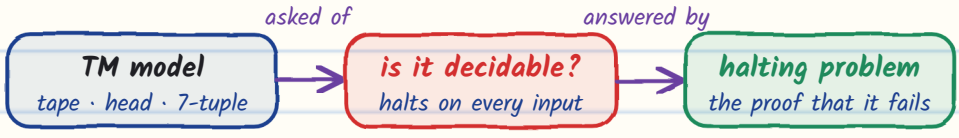
$TM = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$	$\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R, S\}$
$\delta(q, x) = (q', y, D)$	head scans the **first symbol of v**
$ID = (u, q, v)$ — left, state, right	$\vdash$ = one move · $\vdash^*$ = zero or more
Tape infinite **both** directions	Q, Σ, Γ finite · $B \notin \Sigma$
$\{a^n b^n\}$ machine: $q_0..q_4$, marks X, Y	accept on $\delta(q_3, B) = (q_4, B, R)$
Multi-tape = same power, ≤ quadratic	Offline TM = read-only input + work
LBA = nondet TM, tape = $\|w\| + 2$	LBA accepts the **CS** languages
Church-Turing: TM-computable ⇔ computable	Decidable halts always; semi on yes only
Halting problem **undecidable** (Turing, 1936)	...but it **is** recursively enumerable
$D(M)$: if $H(M,M)$ halts, loop; else halt	Blank-tape halting ≤ halting
Rice: non-trivial property of **L(M)**	Non-trivial = some yes, some no
RE closed: ∪, concat, *, ∩	RE **not** closed under complement
L and L' both RE ⇒ L recursive	Halting is RE but not Co-RE
Regular ⊂ CF ⊂ CS ⊂ Recursive ⊂ RE	$\{a^n b^n c^n\}$ = CS, not CF

→ Top 12 one-liners for MCQs

1. A TM is a **7-tuple**; a PDA 5, a CFG 4
2. Configuration = **(left, state, right)**
3. The head's cell starts **v**, never ends u
4. Halting problem = **undecidable** (Turing, 1936)
5. Halting problem = **RE** — yes-side only
6. Multi-tape ≠ more powerful, only faster
7. LBA = bounded-tape nondet TM ⇒ **CS**
8. Church-Turing: TM = algorithm
9. A ≤ B makes B undecidable when A is
10. Rice is about **L(M)**, not the machine
11. RE ∩ RE = RE, but RE' **not** RE
12. L and L' both RE ⇒ **recursive**

Top traps "Multi-tape is more powerful" is the wrong option · the general δ allows **S** (stay), not just L and R · splitting an ID as (01, q, 011) is wrong · undecidable ≠ slow, it means **no algorithm exists** · Rice asks about the **language**, not the diagram · the $a^n b^n$ table as usually printed has **no rejecting transition**.

All tricks together "Q, Σ, Γ, δ, q_0, B, F" · "left · state · right, head belongs to the right" · "mark, hunt, match, rewind" · "decider = two-way, recogniser = one-way" · "predict, then do the opposite" · Rice asks about L(M).



power is never the question — decidability is

one model, one question, one famous negative answer

✓ Practice MCQs (exam pattern, 10 marks)

try first, then check

1. A Turing machine has how many parts?
(a) 5 (b) 7
(c) 8 (d) 4

2. In δ, the head may move:
(a) L only (b) L or R
(c) L, R or S (d) either way

3. In (u, q, v) the head scans the:
(a) end of u (b) middle of v
(c) a blank cell (d) first of v

4. A multi-tape TM recognises:
(a) the same languages (b) more languages
(c) only regular ones (d) nothing decidable

5. The halting problem is:
(a) decidable, slow (b) by simulation
(c) undecidable, r.e. (d) regular

6. Rice's theorem concerns properties of:
(a) the machine (b) L(M)
(c) the states (d) the tape

7. A nondet TM bounded to |w|+2 is:
(a) a multi-head TM (b) an offline TM
(c) a finite automaton (d) an LBA

8. RE languages are closed under:
(a) complement (b) ∪, concat, *, ∩
(c) intersection only (d) none of these

9. L and its complement both RE ⇒ L is:
(a) only regular (b) not RE
(c) recursive (d) context-free

10. $\{a^n b^n c^n \mid n \geq 1\}$ belongs to:
(a) context-sensitive (b) regular
(c) context-free (d) no class

Answer key → 1-b · 2-c · 3-d · 4-a · 5-c · 6-b · 7-d · 8-b · 9-c · 10-a

Written question Prove by contradiction that the halting problem is undecidable. Then reduce blank-tape halting to it, and name two more problems that inherit undecidability. 10 marks

Model answer ✍

1. **State it and assume a decider H** 2 — H(M, w): does M halt on w? Assume H is right on every pair.
2. **Build D** 2 — D(M) asks H(M, M) and does the **opposite**.
3. **Contradict** 2 — on D(D), D does the other thing than H said.
4. **Reduce** 2 — M' writes w on its own blank tape, then simulates M(w); blank-tape halting ≤ halting, undecidable.
5. **Two more** 2 — state-entry (make q_5 the halt state) · TM equivalence (simulate M then N).

→ Before the exam ✓

- ✓ Can write the 7-tuple and δ format from memory
- ✓ Can split and join an instantaneous description
- ✓ Can reproduce the halting proof in five steps
- ✓ Can state Rice's theorem about L(M), not the machine